

KGRAG: Federated Knowledge Graph Retrieval Across Heterogeneous Domains Without Language Model Inference

Eric G. Suchanek, PhD
Flux-Frontiers, Liberty Township OH

<https://github.com/Flux-Frontiers/KGRAG>

Abstract

KGRAG is a federation and orchestration layer for structural knowledge graphs derived from heterogeneous source domains: Python source code (CodeKG), natural language documentation (DocKG), and scientific or domain-specific corpora (MetaKG). Each backend constructs its knowledge graph from the formal structure of its source—abstract syntax trees for code, document section and reference graphs for text, domain ontologies for scientific data—without any language model participation in graph construction.

The system is organized into five explicit abstraction layers: source parsing, graph storage, semantic indexing, a domain adapter protocol, and a federation orchestrator. A uniform adapter interface (**KGAdapter**) decouples the orchestrator from all backend-specific logic. A single query fans out to all registered knowledge graphs simultaneously; results carry full provenance—node identifier, source path, and relevance score—and are globally ranked by score before return.

The design treats the structurally derived graph as ground truth and uses semantic embeddings strictly as an acceleration layer for locating entry points into that graph. Graph traversal, ranking, and snippet extraction are deterministic: identical inputs produce identical outputs, and no component of the structural pipeline produces probabilistic results. Queries over codebases of several hundred thousand lines typically complete in under one second on commodity hardware.

An MCP server exposes the full federated query surface to any MCP-compatible AI agent, providing grounded, traceable context without requiring the agent to maintain knowledge of individual backend interfaces or repository layouts.

1 Introduction

Large software systems accumulate knowledge in incompatible forms. Source code encodes behavior and architecture in a machine-readable but semantically dense notation. Documentation encodes intent, rationale, and usage patterns in prose. Domain corpora—pathway databases, ontologies, financial models—encode facts in formats specific to each field. These representations are maintained separately, queried separately, and surfaced separately; an analyst or an AI agent that needs to understand a system must traverse all three and reconcile the results manually.

The standard answer is retrieval-augmented generation: embed all documents in a vector index and retrieve the closest matches to a query. This approach treats source code and scientific data as text, discarding the formal structure that makes those artifacts tractable. A Python call graph cannot be derived from a vector search; neither can a metabolic pathway traversal. Vector retrieval approximates relevance; it does not expose structure.

An alternative, exemplified by GraphRAG [1], is to use a language model to extract a knowledge graph from unstructured text. This recovers some structure, but the structure is inferred rather than derived:

entity extraction is probabilistic, edges may be hallucinated, and the resulting graph cannot be independently verified against any formal source.

KGRAG takes a different approach. It builds knowledge graphs from the formal structure of each source domain—abstract syntax trees for code, section and reference graphs for documentation, ontological relations for scientific data—using deterministic parsers with no language model involvement. The graphs are correct by construction. Semantic embeddings are added on top, not instead, to enable natural-language entry points into the structural record.

The contribution of this paper is not a new graph construction algorithm but a new level of abstraction above existing single-domain tools: a uniform federation layer that makes heterogeneous structural knowledge graphs queryable as a single unified corpus.

2 Design Principles

Six principles govern the design of KGRAG and its backend modules.

1. **Derived structure is authoritative.** Knowledge graphs are extracted from formal sources—ASTs, document parse trees, ontological schemas—by deterministic programs. The graph is not a model of the source; it *is* the source, represented relationally. Embeddings are derived from it and are disposable.
2. **Semantics accelerate; structure decides.** Vector search locates entry points. Graph traversal determines what is returned. The two phases are explicit, separable, and independently tunable. A change to the embedding model affects which seeds are selected; a change to the hop count or edge filter affects traversal depth. Neither affects the structural layer.
3. **Every result is traceable.** Every node carries a stable identifier encoding its origin: file path and qualified name for code nodes, document path and section offset for documentation nodes. Every snippet is extracted from a verified location. Nothing enters a result without a source address.
4. **Determinism over approximation.** Given identical inputs, the system produces identical outputs. Ranking is computed by a deterministic composite key. There are no stochastic components in the structural or traversal layers.
5. **Generality through protocol.** The federation layer depends on an abstract adapter interface, not on any specific backend. Adding a new knowledge domain requires implementing five methods; the orchestrator, registry, CLI, and MCP server require no changes.
6. **Independence from language models.** The build pipeline, query engine, graph traversal, ranking, snippet extraction, and analysis tools all run locally without any language model call. A downstream language model may consume the output, but it plays no role in producing it.

3 Architecture Overview

The system is organized into five abstraction layers, each with a single responsibility and depending only on layers below it. Figure 1 shows their arrangement.

Layers 1–3 are implemented independently by each knowledge graph module and are hidden behind the Layer 4 boundary. Layer 5 operates exclusively through the `KGAdapter` interface and has no knowledge of any module’s internal representation.

4 Layer 1: Source Parsing

Each knowledge graph module derives its graph from the formal structure of its source domain. Three parsers are currently implemented.

Layer 5: Federation KGRAG orchestrator, KGRegistry, MCP server
Layer 4: Adapter Protocol KGAdapter: <code>is_available</code> , <code>query</code> , <code>pack</code> , <code>stats</code> , <code>analyze</code>
Layer 3: Semantic Indexing LanceDB vector store, sentence-transformer embeddings
Layer 2: Graph Storage SQLite, typed nodes and edges, BFS traversal, provenance
Layer 1: Source Parsing AST (code), document parse tree (docs), ontological schema (domain)

Figure 1: KGRAG abstraction layers. Each layer depends only on those below it. The adapter protocol at Layer 4 is the boundary between domain-specific and domain-agnostic components.

4.1 Python Codebase Parsing (CodeKG)

`CodeGraph.extract()` performs two deterministic AST passes over all `.py` files in a repository. Pass 1 extracts definitions and emits structural edges: `CONTAINS` (module to class, class to method), `IMPORTS` (module-level import relationships), and `INHERITS` (class to base class). Pass 2 extracts the call graph, emitting `CALLS` edges annotated with call-site evidence: the line number and expression text of each call site, extracted directly from the AST node.

A data-flow visitor (`CodeKGVisitor`) extends `ast.NodeVisitor` to emit three additional edge types per function: `READS` (variable consumption), `WRITES` (variable assignment), and `ATTR_ACCESS` (attribute access on a named object). These capture intra-function data movement without requiring execution or type inference.

A post-build step, `resolve_symbols()`, name-matches every unresolved call target (represented as a `sym: stub` node) against all first-party definitions and writes `RESOLVES_TO` edges. This enables cross-module fan-in queries: given a function, which functions in other modules call it?

All nine edge relations (`CONTAINS`, `CALLS`, `IMPORTS`, `INHERITS`, `RESOLVES_TO`, `READS`, `WRITES`, `ATTR_ACCESS`, and module-level `NEXT`) are derived from syntax. No relation is inferred.

4.2 Document Corpus Parsing (DocKG)

DocKG parses Markdown corpora into a heterogeneous graph. Six node kinds are extracted: `document` (the file itself), `section` (a heading and its immediate content), `chunk` (a paragraph-level unit of text), `entity` (a named concept), `keyword` (a significant term), and `topic` (a thematic cluster). Eight edge types connect them.

Structural edges follow the document hierarchy: `CONTAINS` (document to section, section to chunk), `NEXT` (sequential chunk ordering), and `REFERENCES` (explicit hyperlinks between documents). Semantic edges are derived from co-occurrence and overlap analysis: `SIMILAR_TO`, `HAS_TOPIC`, `MENTIONS_ENTITY`, `HAS_KEYWORD`, and `CO_OCCURS_WITH`. Semantic edges are computed from document content, not from a language model; the same parser run on the same corpus produces the same graph.

4.3 Domain Corpus Parsing (MetaKG)

MetaKG wraps a domain-specific backend—initially targeting metabolic pathway databases—via the same `KGAdapter` interface. The parsing strategy is domain-defined; the interface contract is uniform. A biochemical pathway graph derived from KEGG or a legal ontology derived from regulatory filings both surface identically to the federation layer.

5 Layer 2: Graph Storage

All three modules persist their graphs to SQLite in a two-table schema: a **nodes** table and an **edges** table. Node records carry a stable string identifier, a kind, a name, and a JSON metadata blob. Edge records carry source node ID, relation name, destination node ID, and optional JSON evidence.

Node identifiers are deterministically constructed from kind, source path, and qualified name. For example: `fn:src/kg_rag/orchestrator.py:KGRAG.query`. This means a node ID is stable across rebuilds as long as the source does not move, and it is human-readable and directly linkable to the source.

The key traversal primitive is `expand(seed_ids, hop, rels)`: a bounded breadth-first search from a set of seed nodes, following a specified set of edge relations up to a configurable hop limit. Each reached node is annotated with `ProvMeta`: its minimum hop distance from any seed and the originating seed node ID. Provenance travels with every result.

A complementary primitive, `callers_of(node_id)`, performs a two-phase reverse traversal: first collecting direct incoming `CALLS` edges, then following `RESOLVES_TO` edges through `sym` stubs to find cross-module callers. The two phases are explicit; nothing is inferred.

6 Layer 3: Semantic Indexing

`SemanticIndex` reads nodes from a `GraphStore`, constructs a canonical text document for each node (name plus available descriptive text), embeds them using a sentence-transformer model, and stores the vectors in LanceDB. The default model is `all-mpnet-base-v2` (768-dimensional embeddings), overridable per deployment.

The index is derived from the SQLite graph and is fully disposable. Deleting and rebuilding it does not affect the structural record. The only function of the vector index is to rank nodes by semantic similarity to a query string, producing a short list of seed node IDs for the subsequent structural traversal.

The separation is intentional and load-bearing. If the embedding model is replaced, the new index will select different seeds; the traversal from those seeds will still follow the same structural edges and produce structurally correct results. The accuracy of the structural layer does not depend on the quality of the embedding model.

7 Layer 4: The Adapter Protocol

`KGAdapter` is an abstract base class with five abstract methods. Every knowledge graph module exposes itself to the federation layer through this interface and through no other.

```
class KGAdapter(ABC):
    def __init__(self, entry: KGEntry) -> None: ...

    @abstractmethod
    def is_available(self) -> bool: ...

    @abstractmethod
    def query(self, q: str, k: int = 8) -> list[CrossHit]: ...

    @abstractmethod
    def pack(self, q: str, k: int = 8,
             context: int = 5) -> list[CrossSnippet]: ...
```

```

@abstractmethod
def stats(self) -> dict[str, Any]: ...

@abstractmethod
def analyze(self) -> str: ...

```

`is_available()` tests whether the underlying library is installed and the database is built. The orchestrator calls this before dispatching any query; an unavailable adapter is silently skipped (or, in strict mode, raises an error).

`query()` executes the module’s hybrid query pipeline and returns a list of `CrossHit` objects. `CrossHit` is a domain-neutral dataclass:

```

@dataclass
class CrossHit:
    kg_name: str          # registered KG instance name
    kg_kind: KGKind       # CODE | DOC | META
    node_id: str          # stable source-specific identifier
    name: str             # human-readable node name
    kind: str             # node kind within its domain
    score: float          # relevance (higher is better)
    summary: str          # docstring, heading, or description
    source_path: str      # file or document path

```

`pack()` executes the query and additionally extracts source-grounded snippets, returning `CrossSnippet` objects that carry the raw content alongside the score and provenance fields.

`stats()` returns basic graph statistics (node count, edge count). `analyze()` returns a Markdown-formatted analysis report generated by the module’s own analysis tooling—a thorough structural and semantic characterization of the knowledge graph.

The adapter protocol defines the boundary between the universal and the specific. Everything above Layer 4 is domain-agnostic.

8 Layer 5: Federation

8.1 Registry

`KGRegistry` maintains a persistent SQLite table of `KGEntry` records. Each entry records the KG instance name, its kind, the repository root, paths to its SQLite and LanceDB databases, the module version, and a JSON metadata blob. Registration is performed once per project; subsequent queries consult the registry to locate all known KG instances.

The registry is file-local by default (`~/.kgrag/registry.sqlite`), giving each user a private, unified view of all their registered knowledge graphs regardless of where they reside on the file system. A hosted registry variant provides team-shared coordination without requiring shared file system access.

8.2 Orchestrator

`KGRAG` is the cross-KG orchestrator. On each query, it:

1. Retrieves all entries from the registry, optionally filtered by kind.
2. For each entry, instantiates (or retrieves from cache) the corresponding `KGAdapter` and verifies availability.

3. Dispatches the query string to each available adapter in sequence, collecting `CrossHit` or `CrossSnippet` lists.
4. Merges all results into a single list and sorts by score descending.
5. Returns a typed result object (`CrossQueryResult` or `CrossSnippetPack`) with the merged list and per-KG metadata.

The dispatch loop is deliberately simple. Individual adapter calls are independent; a failure in one does not abort the others. In permissive mode (the default), failing adapters are skipped; in strict mode, failures propagate.

Global ranking is by score only. Each adapter returns scores on a consistent scale; the orchestrator does not re-normalize. This is adequate for the current use case and avoids the assumptions required for more sophisticated score fusion.

8.3 Result Types

`CrossQueryResult` carries the merged hit list, the per-KG breakdown, the total hit count, and the number of KGs that were actually queried. The per-KG breakdown preserves each adapter’s original ranked order for inspection.

`CrossSnippetPack` carries the merged snippet list, the number of KGs queried, and an approximate token count for budget-aware context assembly. `CrossSnippetPack.render()` produces a single LLM-ready string with section headers identifying each snippet’s source domain, KG name, file path, and line span.

9 Query Execution

A federated query executes as follows, at each participating adapter:

1. **Semantic seeding.** The query string is embedded and used to retrieve k seed nodes from LanceDB. Seeding is approximate; it identifies nodes whose descriptive text is closest to the query in embedding space.
2. **Structural expansion.** `GraphStore.expand()` performs BFS from the seed IDs, following a specified set of edge relations up to a configurable hop limit. Each reached node is annotated with `ProvMeta`: minimum hop distance and originating seed.
3. **Ranking.** Nodes are ranked by a deterministic composite key: hop distance (ascending), seed embedding distance (ascending), node kind priority, and node ID as a tiebreaker.
4. **Deduplication.** For snippet queries, nodes are deduplicated by file and source span. A node whose span overlaps an already-retained span in the same file is discarded.
5. **Snippet extraction.** Source text is extracted at each retained node’s verified path and line span, with a configurable context window.

After each adapter completes steps 1–5, the orchestrator merges the resulting `CrossHit` or `CrossSnippet` lists and performs a final global sort by score. The global sort is the only operation that the orchestrator performs on content; it does not re-rank, re-score, or filter.

End-to-end latency for a federated query over a codebase of ~ 100 k lines and a documentation corpus of ~ 800 nodes is typically under one second on commodity hardware (Apple M-series or Intel Core i7). The dominant cost is the embedding of the query string; structural traversal and SQLite lookups are negligible by comparison.

10 Snapshot System

Knowledge graphs change as their source corpora evolve. The snapshot system captures `SnapshotMetrics` at each commit, keyed by the pre-commit tree hash produced by `git write-tree`. Using the tree hash rather than the commit hash ensures that the snapshot corresponds precisely to the state of the files being committed, even before the commit object exists.

A `Snapshot` record carries the tree hash, branch name, timestamp, module version, a `SnapshotMetrics` struct (node counts, edge counts, coverage score, issue count), and `SnapshotDelta` fields comparing the current state against the immediately preceding snapshot and against the baseline (oldest recorded snapshot).

Snapshots are stored as JSON files in `.codekg/snapshots/` or `.dockg/snapshots/` alongside a `manifest.json` index. A pre-commit hook captures snapshots automatically. The escape hatch `CODEKG_SKIP_SNAPSHOT=1` (or `DOCKG_SKIP_SNAPSHOT=1`) suppresses capture for a single commit.

The snapshot history provides a longitudinal record of graph evolution: which commits added nodes, which removed them, and how semantic coverage trended over time. For large corpora with multiple contributors, this record is the primary instrument for detecting documentation decay and tracking architectural drift.

11 MCP Integration

An MCP server wraps the KGRAG orchestrator and exposes five federated tools to any MCP-compatible AI agent:

Tool	Function
<code>krag_stats()</code>	Registry summary: KG count, kinds, built status
<code>krag_list([kind])</code>	List registered KG entries, optionally filtered
<code>krag_info(name)</code>	Full detail for a single KG entry
<code>krag_query(q, [k, kinds])</code>	Federated semantic query, JSON result
<code>krag_pack(q, [k, kinds])</code>	Federated snippet pack, Markdown output

Individual module MCP servers (CodeKG’s `codekg mcp`, DocKG’s `dockg mcp`) expose deeper, domain-specific tool sets—call graph traversal, centrality ranking, snapshot comparison for CodeKG; document structure navigation, hot chunk identification for DocKG—for tasks that require module-specific capabilities. The KGRAG MCP server is the first point of contact for cross-domain queries; module servers handle domain-specific deep dives.

The MCP transport is `stdio`, making it compatible with Claude Code, Kilo Code, GitHub Copilot, Cline, and Claude Desktop without any additional networking configuration.

12 Comparison with Inference-Based Graph Construction

GraphRAG [1] is the most widely deployed system in the vicinity of this work. Both KGRAG and GraphRAG augment retrieval with a knowledge graph; the origin and guarantees of that graph differ substantially.

In GraphRAG, a language model extracts entities and relationships from text documents. The graph is a product of language model inference; its correctness is bounded by extraction prompt quality and model comprehension. Errors introduced during extraction are silent: the system has no mechanism to verify that an extracted relationship corresponds to anything in the source.

In KGRAG, knowledge graphs are derived from formal artifacts by deterministic programs. A `CALLS` edge in CodeKG exists because a call expression was found in the AST at the recorded line number. A

REFERENCES edge in DocKG exists because a hyperlink was found in the Markdown source. A metabolic reaction edge in MetaKG exists because it appears in the source schema. No edge is inferred; every edge is verified against a source address.

Table 1 summarizes the key distinctions.

Table 1: Inference-based vs. structure-derived knowledge graph retrieval

Property	GraphRAG [1]	KGRAG
Graph source	LLM extraction from text	Formal source parsing
Graph correctness	Probabilistic	Deterministic
Edge evidence	NL relationship description	Source address + line
LLM required to build	Yes	No
LLM required to query	No	No
Domain generality	Text corpora	Code, text, any schema
Structural edge types	Typed (extracted)	Typed (derived from AST)
Result provenance	Document reference	Node ID + file + line
Hallucination risk	Present in graph construction	Bounded to downstream only
Query latency	Seconds to minutes	Under one second (local)

The two systems are not in competition for the same problem. GraphRAG is designed for global sense-making over large unstructured text corpora: questions like “What are the major themes across this document collection?” GraphRAG requires a language model at construction time, accepts probabilistic accuracy, and optimizes for coverage at scale.

KGRAG is designed for structural navigation of formally specified artifacts: questions like “Which functions in this codebase call the session invalidation path?”, “Which documentation sections reference this API?”, or “Which metabolic pathways share this enzyme?” The answers are precise because the sources are precise. The system makes no accuracy trade-off at construction time because no inference occurs there.

When KGRAG’s output is consumed by a downstream language model, the combination constitutes knowledge-graph-augmented generation in which the knowledge graph was not itself produced by a language model. We term this *Structural KG-RAG* [2]: every piece of context passed to the synthesis stage has deterministic provenance traceable to a source address.

13 Conclusion

KGRAG demonstrates that structural knowledge from heterogeneous source domains can be federated under a single uniform interface without collapsing domain specificity into a common text representation. The adapter protocol cleanly separates domain-specific extraction from domain-agnostic orchestration; adding a new knowledge domain requires implementing five methods and no changes to the orchestration, registry, CLI, or MCP server.

The key architectural decisions are:

- Treat formally derived structure as ground truth and semantics as acceleration.
- Place the provenance record at the node level, not the document level.
- Make every layer independently testable and independently substitutable.
- Keep the language model out of the construction pipeline entirely.

The result is a retrieval system that is fast, auditable, and correct by construction at the structural layer. When coupled with a language model for synthesis, it provides grounded context whose every element can be verified against the source. The correctness guarantee holds regardless of the language model’s own accuracy, because the model receives facts rather than constructs them.

References

- [1] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, and Jonathan Larson. From local to global: A graph RAG approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024. <https://arxiv.org/abs/2404.16130>
- [2] Eric G. Suchanek. CodeKG: Deterministic code navigation via knowledge graphs, semantic indexing, and grounded context packing. Technical report, Flux-Frontiers, 2025. https://github.com/Flux-Frontiers/code_kg
- [3] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of EMNLP 2019*, 2019. <https://arxiv.org/abs/1908.10084>